

The background is a solid red color. A large, faint arch made of small red dots spans the top half of the image, framing the central text.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

The background is a solid red color. Overlaid on this is a large, circular pattern of small white dots. The dots are arranged in concentric circles, with the density of the dots being higher in the center and gradually decreasing towards the outer edges, creating a halftone or dot-matrix effect.

OPERATING SYSTEMS 2025.2



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

ZFSLite: a lightweight ZFS file system implementation

Vũ Nguyễn Anh Tuấn 202417212

Nguyễn Phan Vĩ 202417218

ONE LOVE. ONE FUTURE.



Table of Contents

- I. ZFS overview
- II. ZFS features & implementation analysis
- III. Demo



- File system with volume management capabilities.
- Designed to ensure that data stored on disks cannot be lost.
- Includes a mechanism for dataset and pool-level snapshots and replication, including snapshot cloning
- Is a 128-bit file system

II. ZFS features & implementation analysis

- Pooled Storage
- Copy-on-Write (CoW)
- Data Integrity and Self-Healing
- Snapshots and Clones

Pooled Storage

- Instead of creating rigid partitions on individual disks, ZFS abstracts your physical drives into a shared pool of storage called a *zpool*.
 - Multiple file systems (called "datasets") are created on top of this single pool.
 - Datasets dynamically share the available space

II. ZFS features & implementation analysis

```
1 uint64_t ZPool::alloc_block() {
2     if (members.empty()) return 0;
3
4     // Round-robin across VDevs to stripe allocations
5     for (size_t attempt = 0; attempt < members.size(); ++attempt) {
6         size_t idx = (next_vdev_idx + attempt) % members.size();
7         uint64_t local_blk = members[idx].alloc->alloc_block();
8         if (local_blk != 0) {
9             next_vdev_idx = (idx + 1) % members.size();
10            return encode_block((uint16_t)idx, local_blk);
11        }
12    }
13    return 0; // All VDevs are full
14 }
15
16 bool ZPool::read_block(uint64_t global_blk, void* buffer) {
17     uint16_t vdev_idx;
18     uint64_t local_blk;
19     decode_block(global_blk, vdev_idx, local_blk);
20     if (vdev_idx >= members.size()) {
21         std::cerr << "[zpool] read_block: invalid vdev_idx=" << vdev_idx << std::endl;
22         return false;
23     }
24     return members[vdev_idx].vdev->read_block(local_blk, buffer);
25 }
```


II. ZFS features & implementation analysis

Copy-on-Write (CoW)

- Traditional file systems modify data by overwriting the old data in place.
- ZFS takes a different approach.
 - When a change is made, ZFS writes the new data to a completely new, empty block on the disk.
 - Only after the write is fully completed does ZFS update the system's pointers to reference the new block, subsequently freeing up the old one.

=> Guarantees that your data is never left in an incomplete or corrupt state if the system crashes or loses power mid-write, eliminates the need for time-consuming file system checks (fsck) on reboot.

II. ZFS features & implementation analysis

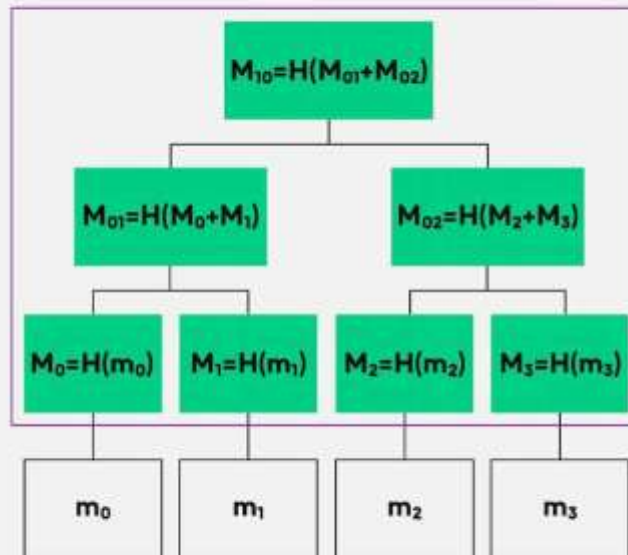
```
1 void ZFS::cow_propagate(std::vector<PathNode>& trace, zfs1_dnode& leaf) {
2     zfs1_dnode cur = leaf;
3     std::vector<uint64_t> deferred_frees;
4
5     // Traverse the trace from bottom (leaf) to top (root)
6     for (int i = (int)trace.size() - 1; i >= 0; --i) {
7         PathNode& node = trace[i];
8
9         // 1. Allocate a completely new block and write the current node state
10        uint64_t new_blk = alloc->alloc_block();
11        write_dnode(new_blk, &cur);
12
13        // 2. Record the old block to be freed later
14        deferred_frees.push_back(node.dnode_blk);
15
16        // 3. If we reached the root, update the Uberblock and stop
17        if (i == 0) {
18            uberblock.root_blk = new_blk;
19            compute_sha256(&cur, sizeof(zfs1_dnode), uberblock.root_hash);
20            break;
21        }
22
23        // 4. Otherwise, update the parent's pointer to reference the new block
24        PathNode& parent = trace[i - 1];
25        uint64_t old_db = node.dirent_blk;
26        uint64_t new_db = alloc->alloc_block(); // CoW the parent's dirent block too
27
28        // ... (read old_db, update dirent pointer to new_blk, write to new_db) ...
29
30        // Update the parent dnode to point to the new dirent block
31        for (int s = 0; s < 10; ++s) {
32            if (parent.dnode.direct[s].blk_no == old_db) {
33                compute_sha256(d_buf, VDEV_BLOCK_SIZE, parent.dnode.direct[s].hash);
34                parent.dnode.direct[s].blk_no = new_db;
35                break;
36            }
37        }
38        deferred_frees.push_back(old_db);
39        cur = parent.dnode; // Move up the tree
40    }
41
42    // 5. Critical Phase: Free old blocks ONLY AFTER the new tree is complete
43    for (uint64_t blk : deferred_frees) {
44        cow_free_block(blk);
45    }
46 }
```

Data Integrity and Self-Healing

- **Isolated Checksumming (Merkle Trees):** Stores cryptographic hashes in a block's parent pointer => Guarantees that misplaced writes/silent disk errors are mathematically impossible to hide.
- **Continuous Read-Time Verification:** Every single time a file is requested, ZFS recalculates the hash of the data being read and compares it against the stored pointer.
- **Instant Redundant Retrieval:** Corruption is detected during a read
=> Intercepts the error and fetches the correct data from a redundant source

II. ZFS features & implementation analysis

MERKLE TREE



Root node: Appended and hashed child node values

Nodes: Appended and hashed child node values

Leaf nodes: Data is hashed

Data

Only the HASHED data (M) is regarded as part of the Merkle Tree but not the data (m) as such.

II. ZFS features & implementation analysis

```
1 int ZFS::read(const char *path, char *buf, size_t size, off_t offset) {
2     // ... (path resolution and boundary checks) ...
3
4     for (uint64_t b = start_b; b <= end_b; ++b) {
5         // 1. Locate the trusted hash from the parent dnode's pointer
6         uint64_t target_blk = dn.direct[b].blk_no;
7         uint8_t* expected_hash = dn.direct[b].hash;
8
9         // 2. Perform raw disk I/O
10        char blk[VDEV_BLOCK_SIZE];
11        vdev->read_block(target_blk, blk);
12
13        // 3. Dynamically compute the SHA-256 hash of the retrieved data
14        uint8_t computed_hash[32];
15        compute_sha256(blk, VDEV_BLOCK_SIZE, computed_hash);
16
17        // 4. Validate integrity
18        if (memcmp(computed_hash, expected_hash, 32) != 0) {
19            std::cerr << "SILENT CORRUPTION DETECTED: Hash mismatch on block " <<
20                target_blk << "\n";
21            return -EIO; // Return standard I/O error to the OS
22        }
23
24        // 5. If validation passes, copy data to the user buffer
25        memcpy(buf + buffer_offset, blk + block_offset, copy_length);
26    }
27    return bytes_read;
28 }
```

Snapshots and Clones

- Instantaneous Creation (uses CoW)
- Zero Initial Space Cost (preserves the existing data blocks)
- Immutable
- Writable Clones
 - Efficient Sandboxing (clones only consume space for the *new* changes made to them)

II. ZFS features & implementation analysis

```
1 int ZFS::take_snapshot() {
2     std::lock_guard<std::mutex> lock(zfs_mutex);
3
4     // 1. Record the current state of the Uberblock into the snapshot array
5     uint32_t idx = uberblock.snapshot_count++;
6     uberblock.snapshots[idx].txg = uberblock.txg;
7     uberblock.snapshots[idx].root_blk = uberblock.root_blk;
8     memcpy(uberblock.snapshots[idx].root_hash, uberblock.root_hash, 32);
9
10    // 2. Traverse the entire tree to increment reference counts
11    inc_tree_refcounts(uberblock.root_blk);
12
13    // 3. Persist the allocator and Uberblock state
14    alloc->save();
15    save_uberblock();
16    return 0;
17 }
18
19 void ZFS::inc_tree_refcounts(uint64_t dnode_blk) {
20     zfs_dnode dn;
21     read_dnode(dnode_blk, &dn);
22
23     // Increment the dnode block itself
24     alloc->inc_ref(dnode_blk);
25
26     if (dn.type == ZFSL_DNODE_FILE) {
27         // Increment all direct data blocks
28         for (int i = 0; i < 10; ++i) {
29             if (dn.direct[i].blk_no != 0) alloc->inc_ref(dn.direct[i].blk_no);
30         }
31         // ... (handle indirect blocks) ...
32     } else if (dn.type == ZFSL_DNODE_DIR) {
33         // Traverse directory and recursively increment children
34         // ...
35     }
36 }
```


II. ZFS features & implementation analysis

Tests & Benchmark

```
1. Creating files...
[PASS] create /hello.txt
[PASS] create /world.txt
[PASS] duplicate /hello.txt rejected

2. Single-block write & read...
[PASS] write 4 KB
[PASS] read 4 KB
[PASS] data integrity

3. Multi-block write & read (20 KB)...
[PASS] write 20 KB
[PASS] read 20 KB
[PASS] multi-block data matches

4. mkdir / readdir...
[PASS] mkdir /subdir
[PASS] duplicate /subdir rejected
[PASS] create /subdir/nested.txt

5. Snapshots...
[PASS] take snapshot
[PASS] one snapshot listed
Snapshot txg = 7
[PASS] post-snapshot write is 'Z'
[PASS] restore snapshot
[PASS] restored data is 'A'

5.5 Indirect Blocks (Large File)...
[PASS] create /large.bin
[PASS] write 60KB to /large.bin
[PASS] read 60KB from /large.bin
[PASS] large file data matches exactly (indirect blocks work)
[PASS] truncate /large.bin to 30KB
[PASS] large file size is 30KB
[PASS] unlink /large.bin

5.6 Directory Overflow...
[PASS] mkdir /manyfiles
[PASS] created 20 files in /manyfiles (overflows 1st block)
[PASS] readdir returned exactly 20 files
[PASS] unlink /manyfiles/file_0
[PASS] readdir returned 19 files after unlink
[PASS] file_0 is correctly removed
[PASS] file_19 is still present (swapped successfully)
```

```
6. unlink / rmdir...
[PASS] unlink /subdir/nested.txt
[PASS] /subdir/nested.txt no longer exists
[PASS] rmdir /subdir (now empty)

6.5 Phase 7: Rename, Permissions, GC...
[PASS] create /dir_a/move_me.txt
[PASS] rename (cross-dir) /dir_a/move_me.txt -> /dir_b/moved.txt
[PASS] old file path no longer exists
[PASS] new file path exists
[PASS] chmod correctly updated mode to 0777
[PASS] chown correctly updated uid/gid
[PASS] fsync returned success
[PASS] one snapshot currently exists
[PASS] delete_snapshot succeeded
[PASS] snapshot list is now empty (GC freed blocks)

7. truncate...
Size before truncate: 4096
[PASS] truncate to 100 bytes
[PASS] size is now 100

8. Corruption detection...
[PASS] pre-corruption read OK
Data block number: 38
Corrupted 1 byte in block 38
INTEGRITY ERROR: hash mismatch on block 38
[PASS] corrupted read rejected (hash mismatch)

9. CoW throughput benchmark (100 writes)...
Time:      0.0249163 s
Throughput: 15.6775 MB/s
IOPS:      4013
```

The logo for HUST (Ho Chi Minh University of Science) is displayed in white, bold, uppercase letters. It is centered within a circular pattern of small, light red dots that form a larger, faint circular shape on a dark red background.

HUST

DEMO



HUST

THANK YOU !